Report - Assignment 8

Name: Hima Sai Chandana | Roll: 2401MC37

1. Design Decisions & Key Data Structures

Q1 — Virtual to Physical Address Translation (va2pa)

- **Strategy**: sys_va2pa walks the two-level x86 page table using walkpgdir(pgdir, va, 0).
- **Structures & Macros**: Uses PDX(va) (bits 31–22) and PTX(va) (bits 21–12) to index entries, PTE_ADDR(pte) to clear flags (~0xFFF), and VADDR_OFFSET(va) (va & 0xFFF) for the page offset.
- **Validation**: Returns physical address PTE_ADDR(*pte) | offset only if PTE_P (Present) and PTE_U (User) flags are set; otherwise returns -1.

Q2 — Physical Memory Statistics (getmeminfo)

- **Strategy**: Tracks free memory directly inside kalloc() and kfree() under kmem.lock.
- **Structures**: Shared struct meminfo containing total_pages, free_pages, user_pages (p->sz / PGSIZE), and pt_pages.
- **Page Table Overhead**: count_pt_pages() walks the page directory, counting 1 page for the directory plus 1 for each valid page table.

Q3 — Dumping Process Page Tables (vmprint)

- **Strategy**: Traversing myproc()->pgdir below KERNBASE (0x80000000) to dump virtual-to-physical mappings and flag bits (P, W, U).
- **Kernel & Guard Filtering**: Entries above KERNBASE are aggregated into a single summary count. The stack guard page created by exec() is identified by PTE_P set and PTE_U cleared (0x1).

Q4 — Lazy Memory Allocation (Demand Paging)

- **Strategy**: sys_sbrk(n) ($n > 0$) increments p->sz without allocating physical frames (growproc is kept for $n < 0$).
- **Trap Handling**: On T_PGFLT, trap.c reads %cr2. If $2 \times PGSIZE \leq va < p\text{->}sz$, it allocates a frame via kalloc(), zeroes it, and maps it via mappages() (PTE_W | PTE_U).

- **Edge Cases**: Modified copyuvm() in vm.c to skip unmapped PTEs during fork(). Kernel system call accesses to untouched user buffers trigger demand paging in the trap handler.

2. Problems Faced & Resolutions

Issue	Root Cause	Resolution
PTE to PA Corruption	Included bitwise flags when building physical address.	Applied PTE_ADDR(*pte) mask before ORing page offset.
Race / Counter Mismatch	Non-atomic free_pages tracking in kalloc.c.	Protected free_pages mutations with acquire(&kmem.lock).
copyuvm **Panic on Fork**	copyuvm() assumed all pages up to p->sz were present.	Updated loop to skip !(*pte & PTE_P) entries instead of panicking.
Syscall Faults on Lazy Buffers	Passing unbacked heap memory to read() faulted in kernel mode.	Handled kernel-mode T_PGFLT for valid heap ranges in trap.c.

3. Memory Usage Analysis (Free Pages Before & After)

Q2 — Physical Memory Statistics (memtest)

Execution Phase	Free Pages	User Pages	Delta	Explanation

Initial State	56,750	3	—	Baseline process footprint.
After `sbrk(+16KB)`	56,746	7	-4	Immediate eager frame allocation.
Child Process (`fork`**)**	56,737	7	-9	7 user pages + 2 page table pages duplicated.
After Child Reaped	56,746	7	$+9$	All child frames reclaimed; no leaks.

Q4 — Lazy Memory Allocation (`lazytest`)

Execution Phase	Free Pages	User Pages	Mapped Frames	Delta	Explanation
Initial State	56,750	3	3	—	Baseline process footprint.
After `sbrk(32MB)`	56,750	8,195	3	0	Size increased by 8,192 pages; **0 frames used**.
Touch 3 Pages	56,747	8,195	6	-3	Frame allocated only on demand access.
Syscall Buffer Read	56,746	8,195	7	-1	Kernel fault dynamically

					allocated buffer frame.
After Child Reaped	56,746	8,195	7	+ 5	fork() duplicated only mapped frames (7 frames + 2 PT).